

Java por bloques



Objetivo

Trabajar y repasar los conceptos generales que se han desarrollado durante el curso:

- Algoritmos
- Estructuras de datos
- Programación orientada a objetos
- Persistencia
- Validación de datos
- Documentación profesional

Cada bloque se evalúa de forma independiente.

Enfoque

Este proyecto no consiste solo en "hacer que funcione".

Debes demostrar:

- ✓ Comprensión de algoritmos
- ✓ Uso correcto de estructuras de datos
- ✓ Código limpio y validado
- ✓ Documentación técnica profesional

Estructura del proyecto

```
com.docencia.colecciones
com.docencia.condicionales
com.docencia.algoritmos
com.docencia.cadenas
```

```
com.docencia.numeros
com.docencia.herencia
com.docencia.fecha
com.docencia.regex
com.docencia.ficheros
com.docencia.sqlite
```

Qué tienes que hacer

Implementar todos los métodos que contienen:

```
throw new UnsupportedOperationException("Mensaje descriptivo del
algoritmo");
```

! No modificar:

- nombres de métodos
- firmas
- interfaces

Evaluación

Parte	Peso
Tests	8
Documentación	2

Se trata de que seas capaz de comprender los test, resolver los ejercicios y documentar las APIs **interfaces** siguiendo los patrones que hemos trabajado en clase.

Vamos algunos bloques y servicios

1. COLECCIONES

ListService

filtrarPalabrasPorLongitud

Filtra palabras por longitud mínima.

Algoritmo:

- Recorrer lista
- Evaluar longitud

- Construir nueva lista
-

ordenarNumerosAscendente

Ordena números de menor a mayor.

Algoritmo:

- Comparación entre elementos
 - Intercambio (burbuja o sort)
-

calcularMediaLista

Calcula media aritmética.

Algoritmo:

- Sumar valores
 - Dividir entre tamaño
-

SetService

obtenerElementosUnicos

Elimina duplicados.

Algoritmo:

- Conversión a Set
-

intersectarConjuntos

Obtiene elementos comunes.

Algoritmo:

- Retener coincidencias
-

MapService

contarFrecuenciaPalabras

Cuenta repeticiones.

Algoritmo:

- Map con acumulador
-

obtenerClaveConMayorValor

Devuelve clave con mayor valor.

Algoritmo:

- Recorrer mapa
 - Comparar valores
-

2. CONDICIONALES

IfElseService**clasificarEdad**

Clasifica por rango.

Algoritmo:

- if / else
-

SwitchService**obtenerNombreDia**

Devuelve nombre del día.

Algoritmo:

- switch por número
-

3. ALGORITMOS

BusquedaService**buscarIndiceElemento**

Búsqueda lineal.

encontrarMaximo

Comparación secuencial.

OrdenacionService**ordenarBurbujaAscendente**

Ordenación burbuja.

Algoritmo:

- Comparar pares
 - Intercambiar
-

4. CADENAS

StringService

esPalindromo

Comprueba si es palíndromo.

Algoritmo:

- Limpiar texto
 - Invertir
 - Comparar
-

contarVocales

Cuenta vocales.

Algoritmo:

- Recorrer texto
-

5. NÚMEROS

IntegerService

esNumeroPrimo

Algoritmo:

- Divisibilidad hasta raíz
-

calcularFactorial

Algoritmo:

- Multiplicación iterativa
-

MathService

calcularAreaCirculo

Fórmula: $\pi * r^2$

6. HERENCIA

Conceptos:

- Clase abstracta
 - Herencia
 - instanceof
-

7. FECHAS

 LocalDateService

calcularEdad

Algoritmo:

- Period.between
-

8. REGEX

Validaciones:

- DNI
 - Email
 - Teléfono
-

9. FICHEROS

CsvService

leerRegistrosCsv

Algoritmo:

- Leer líneas
 - split(",")
-

10. SQLITE

ClienteDbService

create

Algoritmo:

- Validar datos
- Insertar

DOCUMENTACIÓN OBLIGATORIA

```
/**
 * Explica qué hace el método.
 *
 * @param parametro descripción
 * @return resultado
 * @throws IllegalArgumentException si error
 */
```

Java REPASO JAVA - DAM
Guía rápida de los conceptos clave

CARACTERÍSTICAS JAVA

- ✓ Orientado a objetos
- ✓ Independiente de plataforma
- ✓ Robusto y seguro
- ✓ Multiplataforma
- ✓ Alto rendimiento
- ✓ Distribuido

EJECUCIÓN DE UN PROGRAMA JAVA

Java → JAVAC → .class → JVM → Máquina Virtual → Sistema Operativo

1. SINTAXIS BÁSICA

```
public class HolaMundo {
    public static void main(String[] args) {
        // Mi primer programa
        System.out.println("¡Hola DAM!");
    }
}
```

2. TIPOS DE DATOS

PRIMITIVOS

- byte: 1 byte, -128 a 127
- short: 2 bytes, -32,768 a 32,767
- int: 4 bytes, -2³¹ a 2³¹-1
- long: 8 bytes, -2⁶³ a 2⁶³-1
- float: 4 bytes, 1.4E-45 a 3.4E38
- double: 8 bytes, 4.9E-324 a 1.7E308
- char: 2 bytes, 'a' hasta 'z' y '\u0000' hasta '\uFFFF'
- boolean: 1 byte, true o false

NO PRIMITIVOS (CLASES)

- String, Integer, Double, Boolean, Character, etc.

3. OPERADORES

Aritméticos

```
+ - * / % ++ --
```

Relacionales

```
< > <= >= == !=
```

Lógicos

```
&& || !
```

Asignación

```
= += -= *= /= %=
```

Terminador

```
condición ? valorSiTrue : valorSiFalse
```

4. ESTRUCTURAS DE CONTROL

IF - ELSE

```
if (condición) {
    // código
} else if (otraCondición) {
    // código
} else {
    // código
}
```

SWITCH

```
switch (variable) {
    case 1:
        // código
        break;
    case 2:
        // código
        break;
    default:
        // código
}
```

5. PROGRAMACIÓN ORIENTADA A OBJETOS

CLASE

```
public class Persona {
    // atributos
    private String nombre;
    private int edad;

    // constructor
    public Persona(String n, int e) {
        nombre = n;
        edad = e;
    }

    // métodos
    public void saludar() {
        System.out.println("Hola, soy " + nombre);
    }
}
```

6. ARRAYS Y COLECCIONES

ARRAYS

```
int[] numeros = new int[5];
numeros[0] = 10;
for (int i = 0; i < numeros.length; i++) {
    System.out.println(numeros[i]);
}
```

7. MANEJO DE EXCEPCIONES

```
try {
    // código que puede fallar
} catch (Exception e) {
    // se ejecuta si ocurre la excepción
    System.out.println(e.getMessage());
} finally {
    // siempre se ejecuta
}
```

8. ENTRADA / SALIDA

CONSOLE

```
Scanner sc = new Scanner(System.in);
System.out.println("Nombre: ");
String nombre = sc.nextLine();
String nombre2 = sc.nextLine();
sc.close();
```

9. FECHAS Y HORAS

LOCALIZABLES

```
LocalDate hoy = LocalDate.now();
LocalTime hora = LocalTime.now();
LocalDateTime fechaHora = LocalDateTime.now();
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
String texto = hoy.format(formatter);
```

10. HILOS (MULTITHREAD)

```
Thread hilo = new Thread() {
    // tarea del hilo
    @Override public void run() {
        // código
    }
};
hilo.start();
```

11. STREAMS

```
List<String> lista = List.of("a", "b", "c");
long cantidad = lista.stream()
    .filter(s -> s.length() > 1)
    .count();
lista.stream()
    .map(String::toUpperCase)
    .forEach(System.out::println);
```

12. JDBC (Base de Datos)

```
String url = "jdbc:mysql://localhost:3306/";
Connection con = DriverManager.getConnection(url, "usuario", "password");
PreparedStatement ps = con.prepareStatement(sql);
ResultSet rs = ps.executeQuery();
while (rs.next()) {
    // procesar datos
}
rs.close(); ps.close(); con.close();
```

CONSEJOS

- Java: Main.java → J compiler
- Java: Hello → J compiler
- Java: Hello → J compiler
- Java: Hello → J compiler

BUENAS PRÁCTICAS

- ✓ Usar nombres significativos
- ✓ Refactorizar código cuando sea necesario
- ✓ Mantener el código limpio y ordenado
- ✓ Comentar el código cuando sea necesario

CONVENCIONES DE NOMBRES

- Clases: PascalCase (MyClass)
- Métodos y variables: camelCase (myMethod)
- Constantes: UPPER_CASE (MY_CONSTANT)
- Paquetes: minúsculas separadas por puntos

RECURSOS

- Documentación oficial: <https://docs.oracle.com/javase/>
- Tutoriales: <https://www.w3schools.com/java/>
- Plantillas: <https://www.jetbrains.com/idea/>
- IDE recomendado: IntelliJ IDEA / Eclipse

VERSIÓN

Java 17

Objetivo final

- ✓ Tests en verde
- ✓ Código correcto
- ✓ Algoritmos bien aplicados
- ✓ Documentación completa

Consejos

1. Haz pasar tests (8 puntos)
2. Mejora documentación (2 puntos)